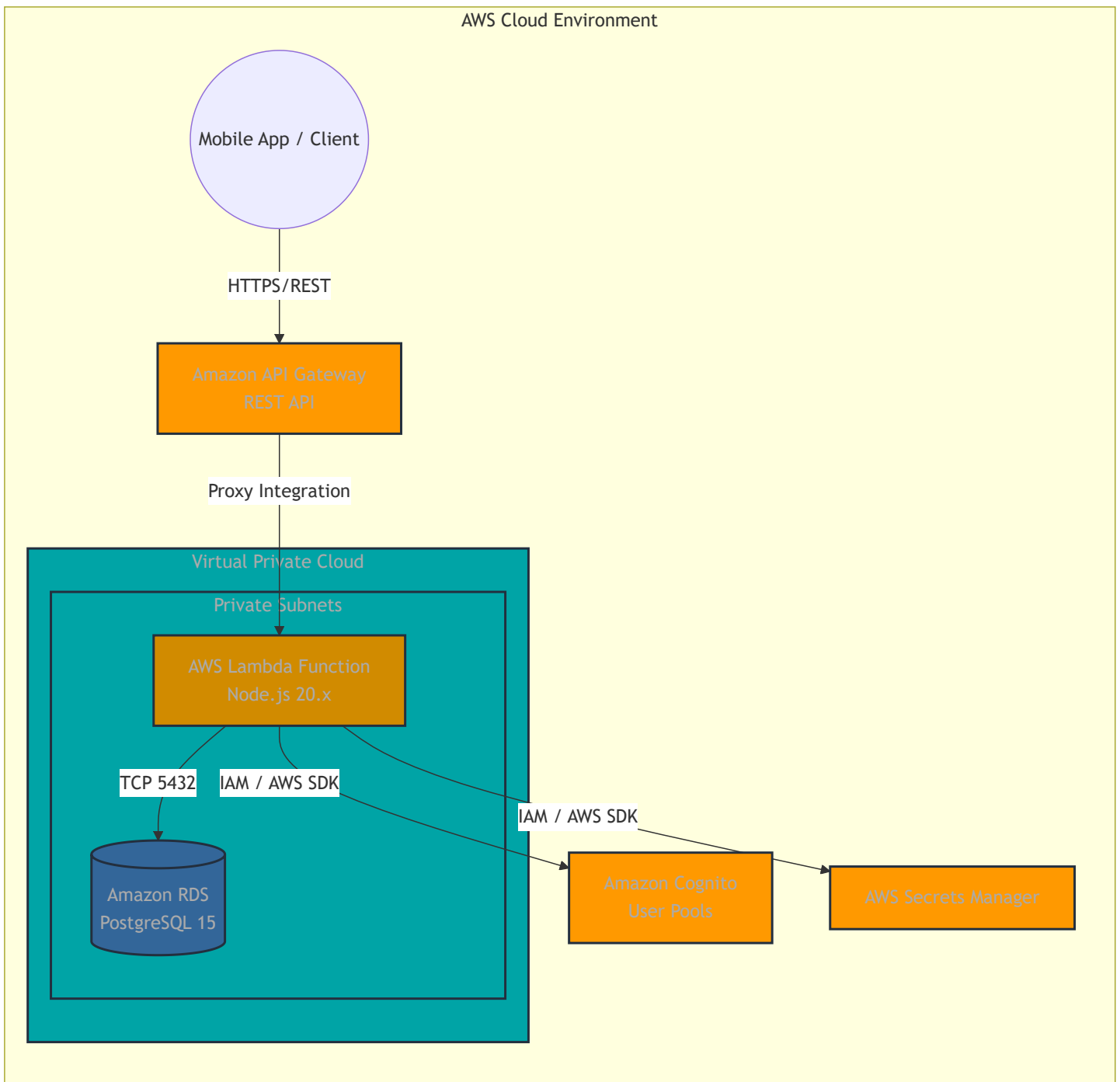


Documentación Técnica de Arquitectura y Diseño - Proyecto Zappi / AWS Backend

Este documento proporciona una visión profunda y técnica de la arquitectura, diseño y herramientas utilizadas en la implementación del backend de **Zappi**. El sistema está diseñado siguiendo principios de **Clean Architecture**, escalabilidad en la nube (Serverless) e Infraestructura como Código (IaC).

1. Visión General de la Arquitectura (Alto Nivel)

El sistema opera bajo un modelo **Serverless** alojado en Amazon Web Services (AWS). Se ha optado por una arquitectura orientada a eventos e integraciones nativas para minimizar el overhead operativo y maximizar la escalabilidad y seguridad.

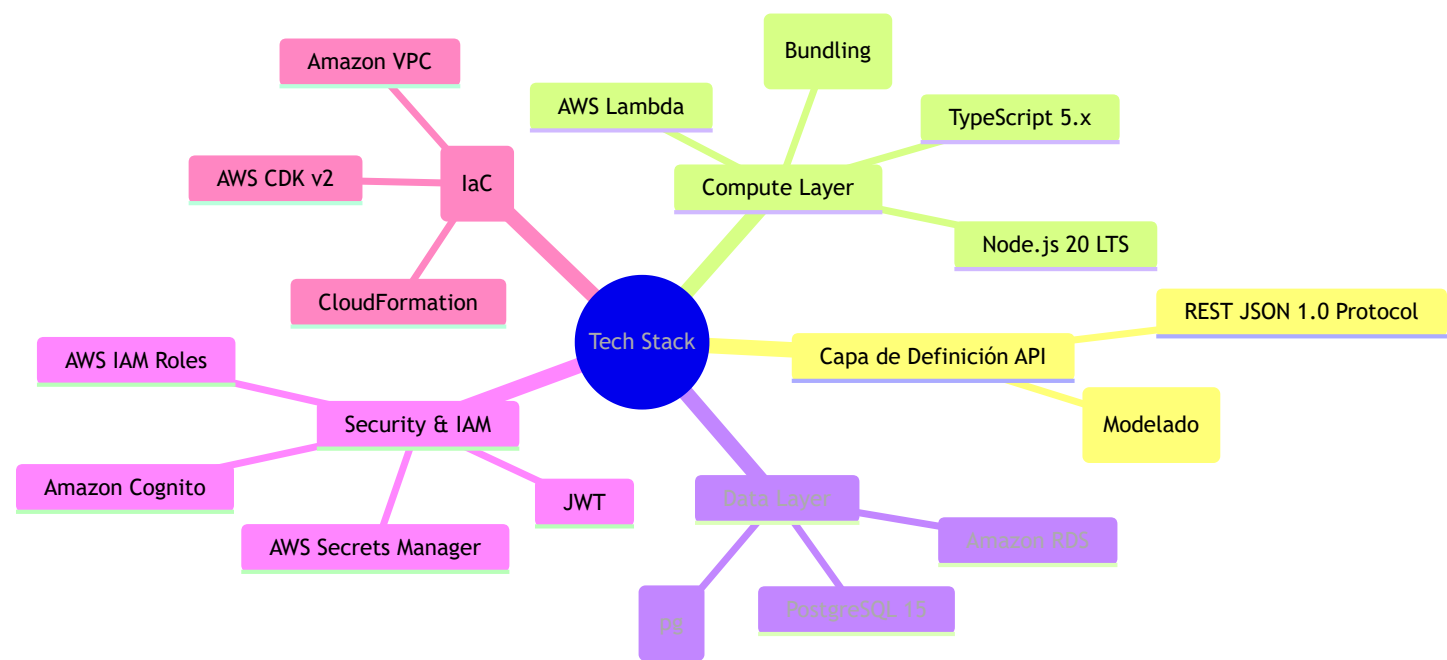


Decisiones Arquitectónicas Clave:

1. **API Gateway + Lambda:** Permite un escalado elástico y pago por uso, absorbiendo picos de tráfico sin comprometer el rendimiento.
2. **Aislamiento en VPC:** Las funciones Lambda y la base de datos RDS se despliegan en subredes privadas. La base de datos no tiene salida ni entrada directa a internet.
3. **Delegación de Identidad:** AWS Cognito actúa como el IdP (Identity Provider), manejando de forma segura políticas de contraseñas, SRP y flujos de autenticación.

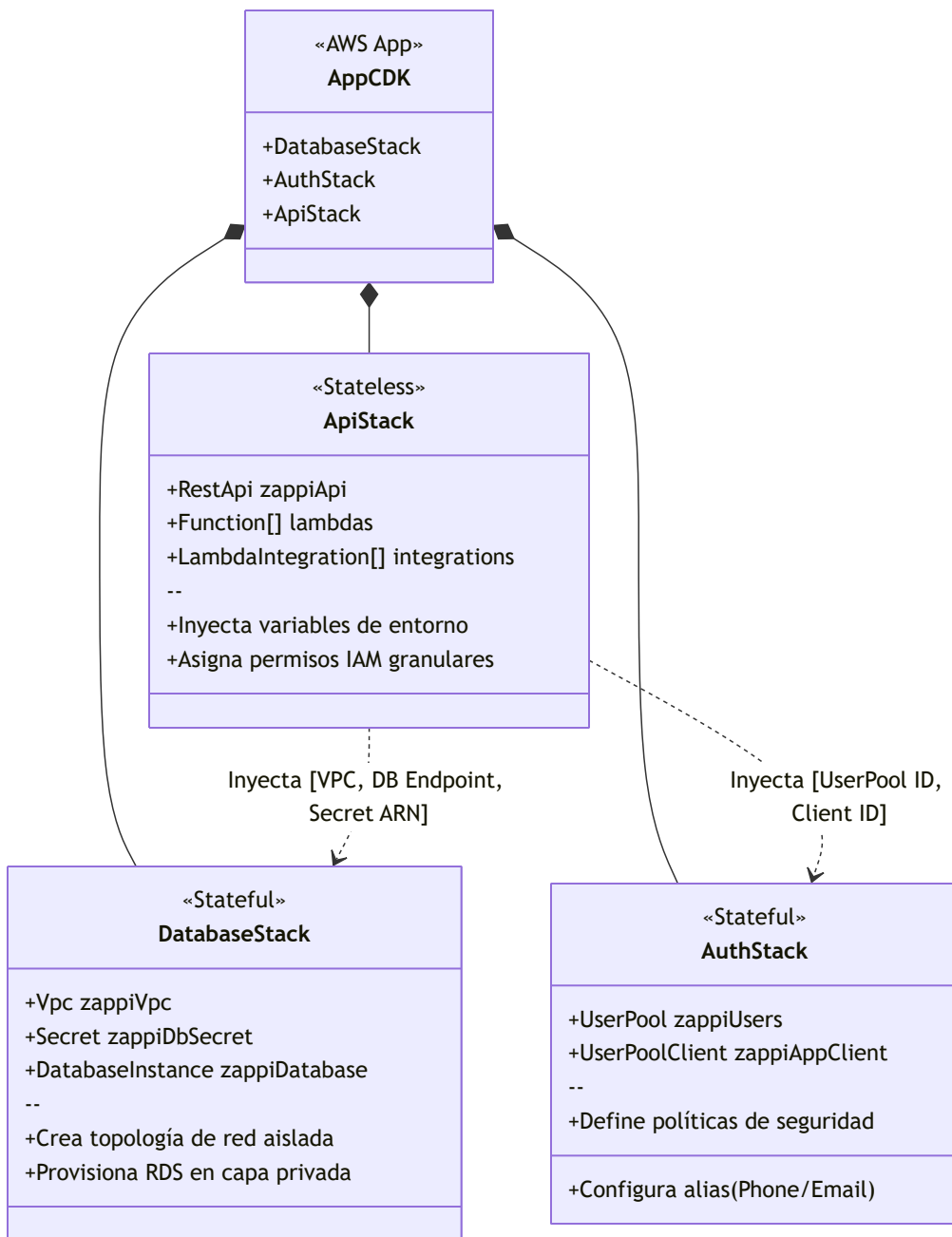
2. Stack Tecnológico y Herramientas

El proyecto hace uso intensivo del ecosistema AWS y el ecosistema moderno de TypeScript, garantizando tipado estático, compilación rápida y despliegues predecibles.



3. Arquitectura de Componentes e Infraestructura (CDK)

La infraestructura está modelada usando **AWS Cloud Development Kit (CDK)**. Se sigue un patrón de diseño estructural mediante la segregación en *Stacks* cohesivos, reduciendo el radio de impacto en los despliegues.



Capas del Backend (Código Fuente):

El código de los Lambdas en `backend/src` sigue una estructura limpia:

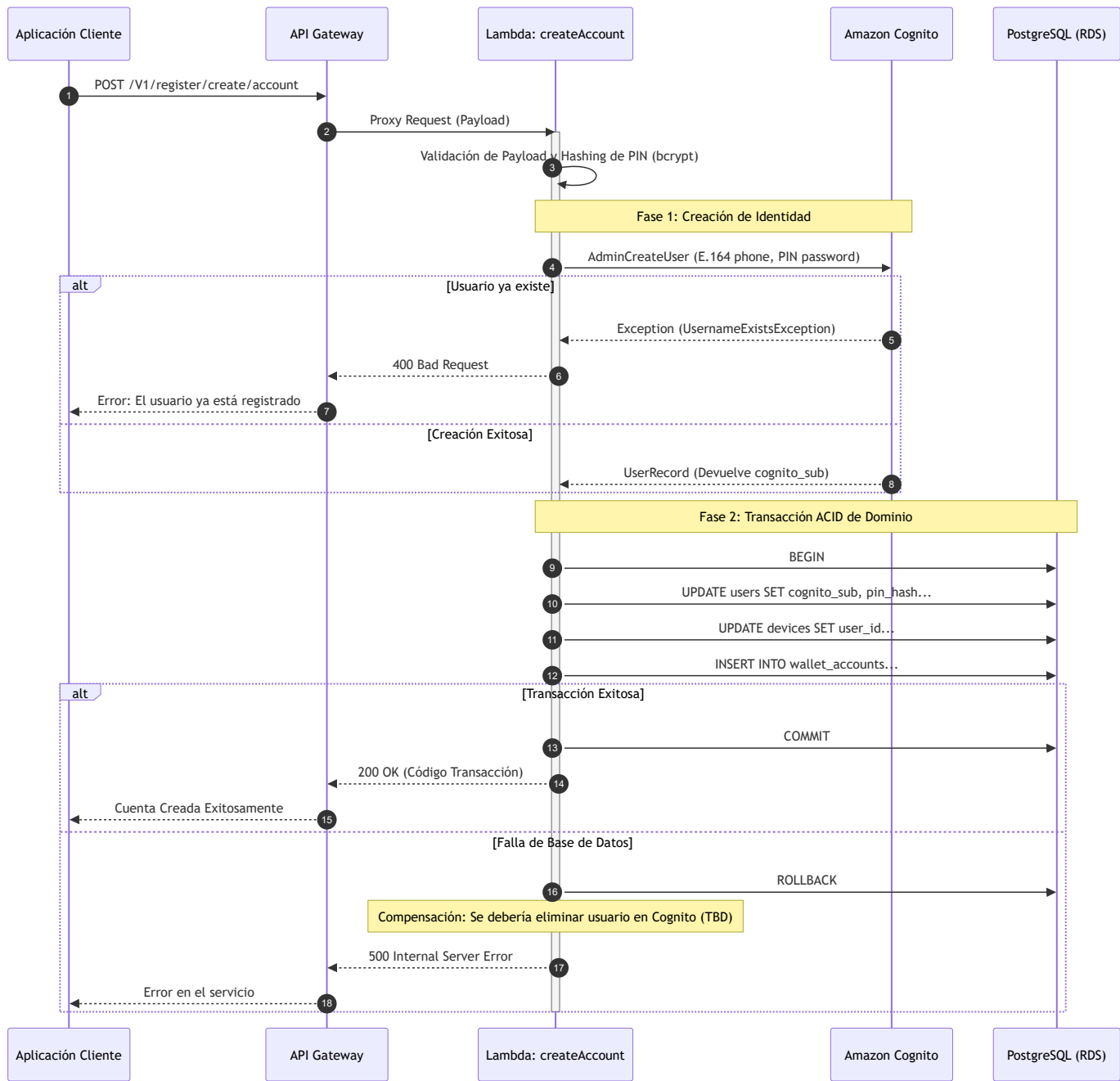
- `handlers/` : Punto de entrada de los lambdas. Orquestación lógica.
- `layers/database/` : Abstracción de conexión a base de datos (Pooler singleton).
- `layers/cognito/` : Interfaz con el SDK v3 de AWS Cognito.
- `utils/` : Respuestas HTTP estandarizadas y utilidades transversales.

4. Diagramas de Secuencia (Flujos Críticos)

A continuación, se detallan los flujos más complejos a nivel transaccional, demostrando la coordinación entre múltiples servicios.

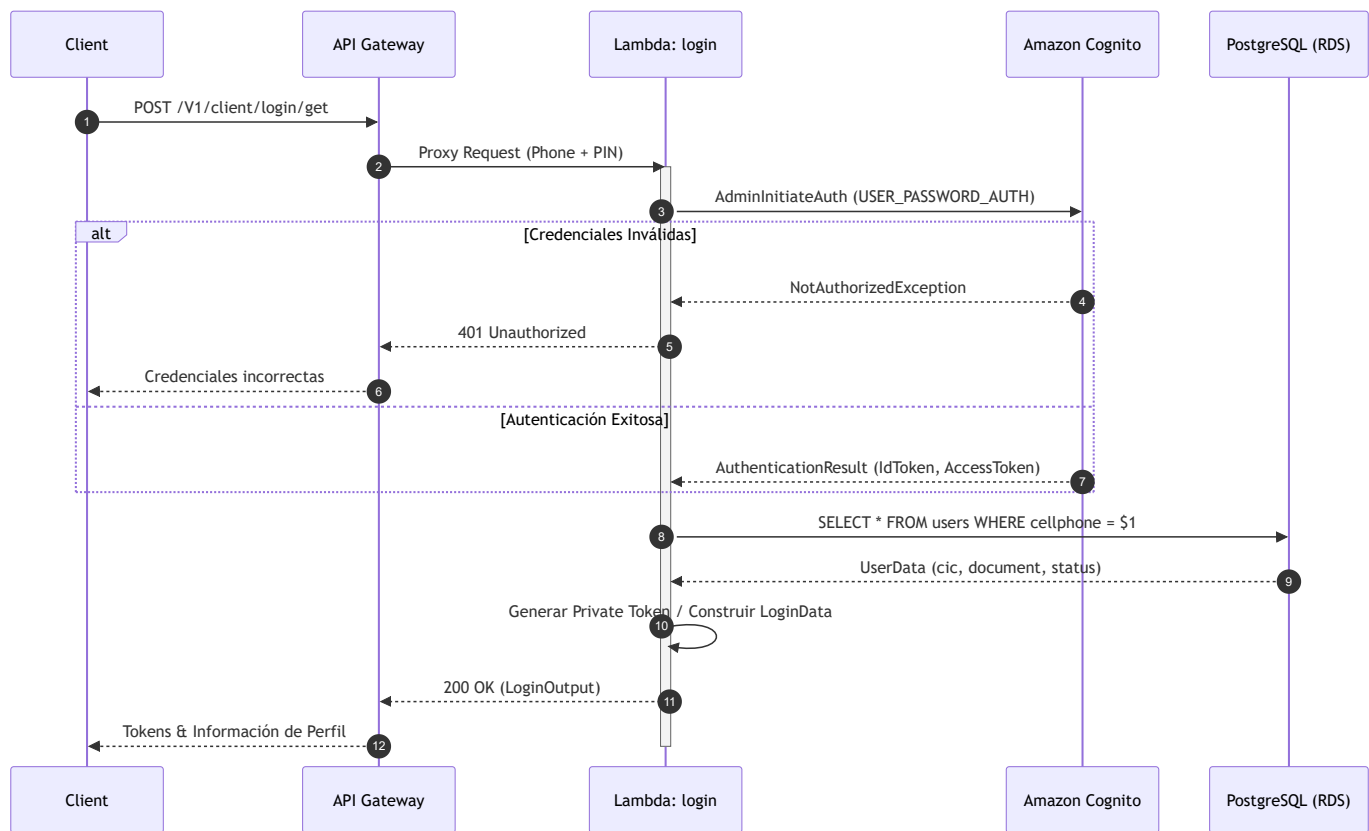
4.1. Flujo de Creación de Cuenta (Sincronización Dual)

Este es un proceso crítico que requiere consistencia eventual entre el proveedor de identidad (Cognito) y nuestra base de datos relacional (RDS).



4.2. Flujo de Autenticación Híbrida (Login)

El sistema valida al usuario, gestiona el estado de sesión y devuelve credenciales temporales o tokens.



5. Prácticas de Ingeniería y Optimización

Rendimiento (Performance)

1. **Gestión de Conexiones a BD:** Se implementa un singleton de validación de pooler de PostgreSQL (`pg.Pool`). Las instancias Lambda "Warm" reutilizan conexiones para reducir drásticamente el tiempo de respuesta (Cold Start mitigation).
2. **Bundling Agresivo:** Se utiliza `esbuild` para compilar el código TypeScript. Esto reduce el tamaño de los artefactos de megabytes a pocos kilobytes, acelerando los tiempos de cold-start de los Lambdas.

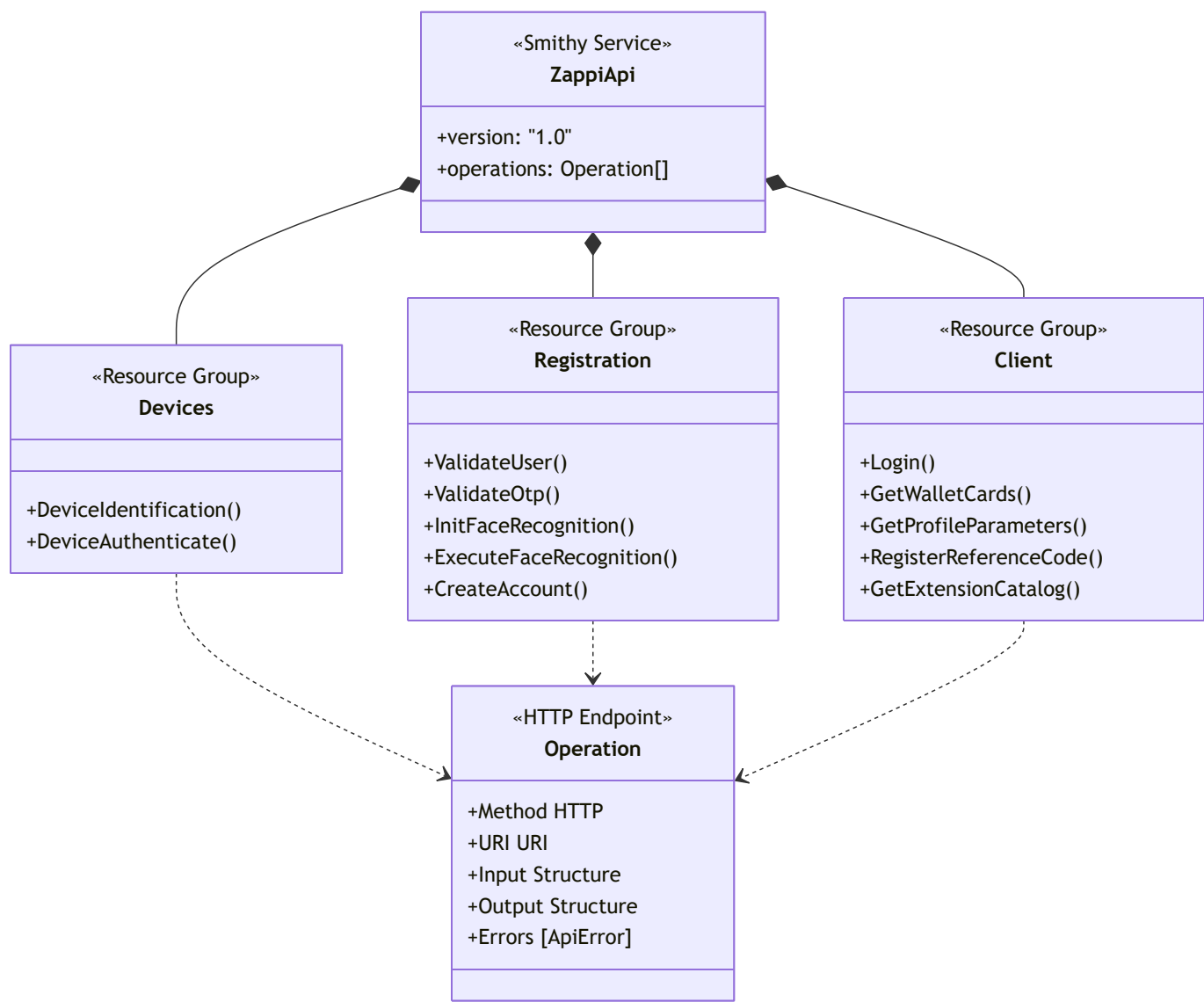
Seguridad Avanzada

1. **Gestión de Secretos:** Las credenciales de la base de datos se generan dinámicamente y se almacenan en **AWS Secrets Manager**. El stack de base de datos de CDK propaga el Secret ARN al stack de la API. Los Lambdas obtienen las credenciales en tiempo de ejecución.
2. **Roles de Ejecución (Least Privilege):** Cada Lambda recibe permisos estrictos y limitados (ej. `cognito-idp:AdminCreateUser` y acceso de lectura estricto al ARN específico en Secrets Manager).

6. Definición de Contratos y API (AWS Smithy)

Se ha utilizado **AWS Smithy** (`backend/smithy/main.smithy`) como la fuente de la verdad para el modelado puramente agnóstico de las APIs. Esto garantiza estructuras inmutables, reutilizables, tipado fuerte y una definición de errores centralizada bajo el protocolo `aws.protocols#restJson1`.

Diagrama de Componentes de la Capa de API



Especificación Técnica de Endpoints

El servicio define 13 operaciones clave, cada una mapeada directamente a una función Lambda independiente:

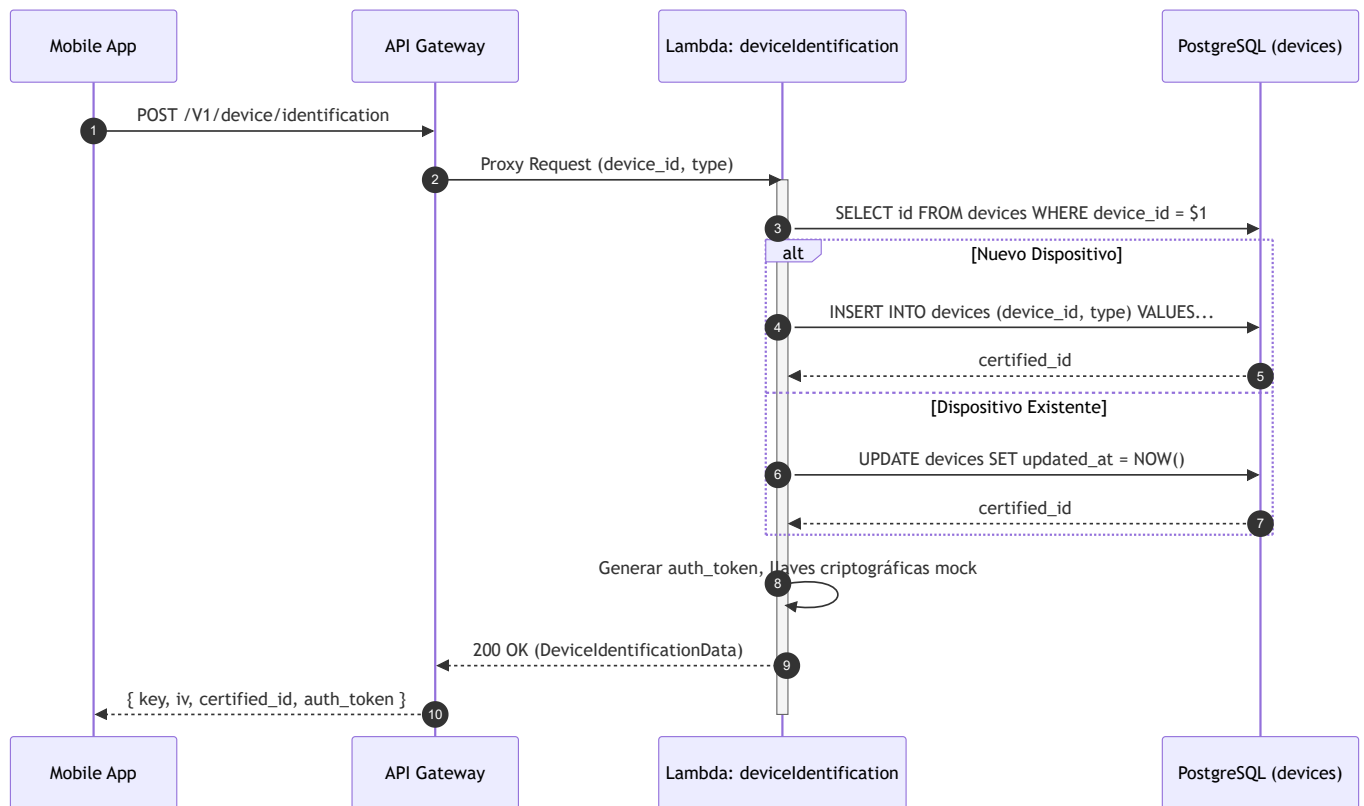
Dominio	Endpoint	Verbo	Propósito Principal	Contexto / Capa
Dispositivos	/V1/device/identification	POST	Identificar y registrar hardware. Retorna llaves de cifrado simuladas y genera ID de dispositivo.	Bootstrapping
Dispositivos	/V1/device/authenticate	POST	Autenticar un dispositivo previamente registrado y verificar trust.	Device Trust
Registro	/V1/register/validate/user	POST	Validar documento de identidad y verificar pre-existencia del usuario en RDS y AS/400.	Onboarding

Dominio	Endpoint	Verbo	Propósito Principal	Contexto / Capa
Registro	/V1/register/validate/otp	POST	Validar el código de seguridad (OTP) enviado por SMS/Email para confirmar posesión de línea.	Onboarding
Biometría	/V1/register/init/face/recognition	POST	Inicializar sesión biométrica (liveness testing). Crea sesión temporal en RDS.	Identity Auth
Biometría	/V1/register/execute/face/recognition	POST	Validar el payload de la biometría (Selfie), firmar el estado y enlazar con el documento.	Identity Auth
Registro	/V1/register/create/account	POST	Endpoint Crítico: Registra en Cognito, asocia información final en RDS, y provisiona una cuenta en Wallet.	Transaccional
Cliente	/V1/client/login/get	POST	Inicia sesión usando Auth Flow (USER_PASSWORD_AUTH) contra AWS Cognito y devuelve perfiles cruzados RDS.	Authentication
Catálogos	/V1/client/device/register/extension/get	POST	Obtiene el catálogo inmutable de extensiones documentales por región.	Metadata
Perfil	/V1/profile/parameters/get	POST	Retorna los parámetros de configuración de UI y mensajes de bienvenida basados en segmentos.	Presentation
Wallet	/V1/client/walletcards/information/get	POST	Devuelve el balance (Core Bancario), movimientos de tarjeta y el PAN enmascarado.	Financial Core
Referidos	/V1/client/reference/register/code	POST	Registra y valida matemáticamente el código promocional de referidos.	Growth
Referidos	/V1/client/reference/welcome	POST	Confirma validez de referidos una vez completado el flujo.	Growth

Diagramas de Secuencia de Operaciones Específicas

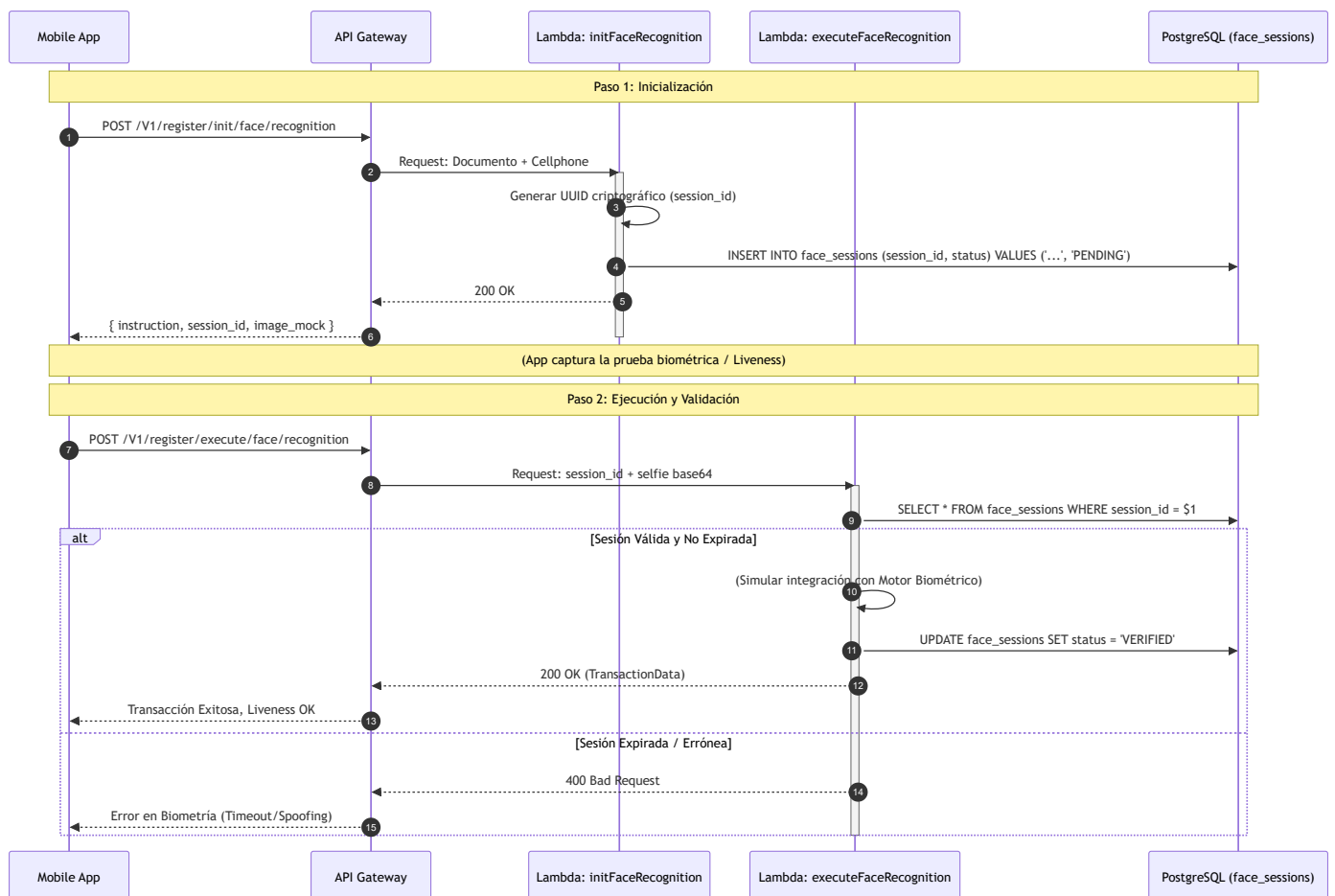
A. Flujo de Identificación de Dispositivos (DeviceIdentification)

El primer paso en la cadena de confianza para vincular hardware (Mobile/Device) con nuestra base de datos, vital para prevención de fraudes y fingerprinting.



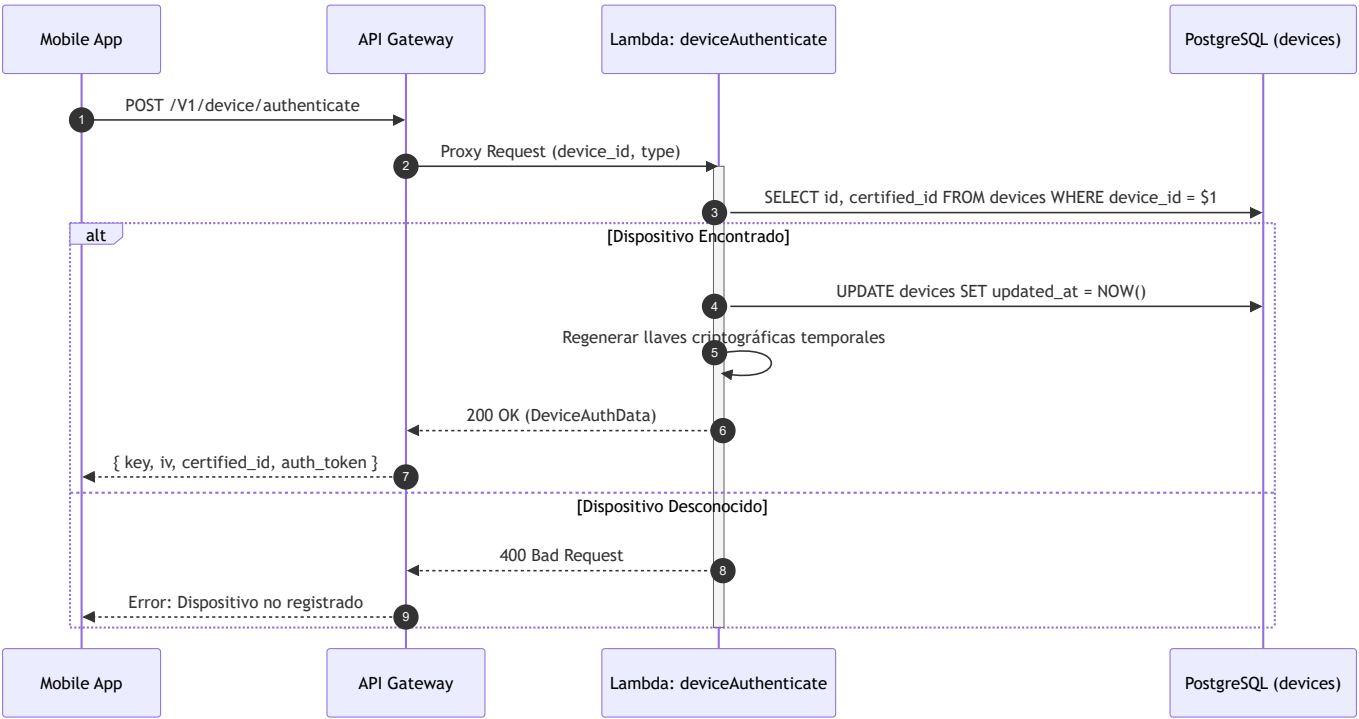
B. Flujo de Reconocimiento Facial (Validación Biométrica en 2 Pasos)

Proceso asíncrono para garantizar pruebas de Liveness sin bloquear los tiempos de procesamiento en cliente.



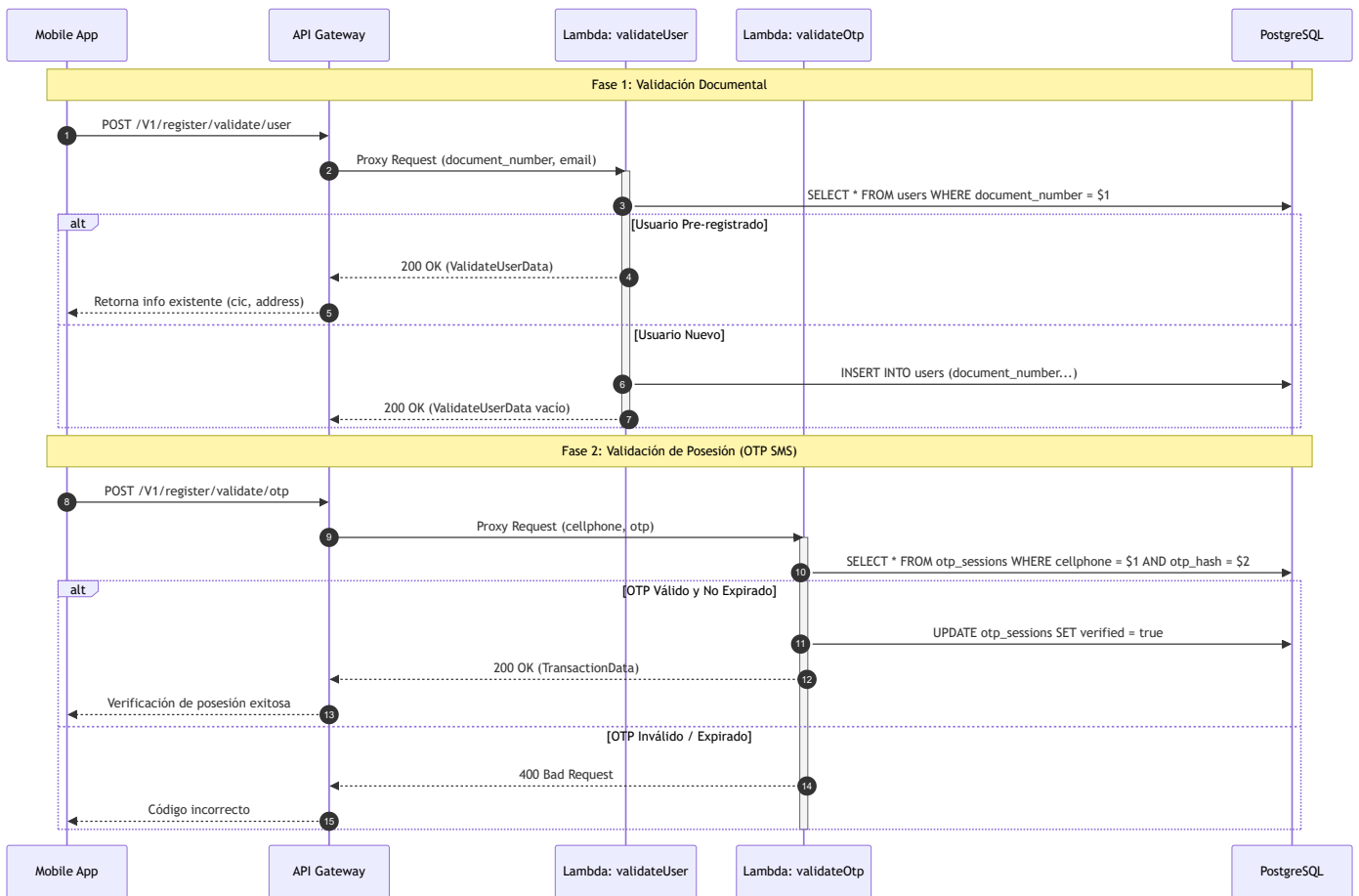
C. Flujo de Autenticación de Dispositivos (DeviceAuthenticate)

Verifica un dispositivo que ya ha pasado por la fase de Identification en inicios de sesión subsecuentes.



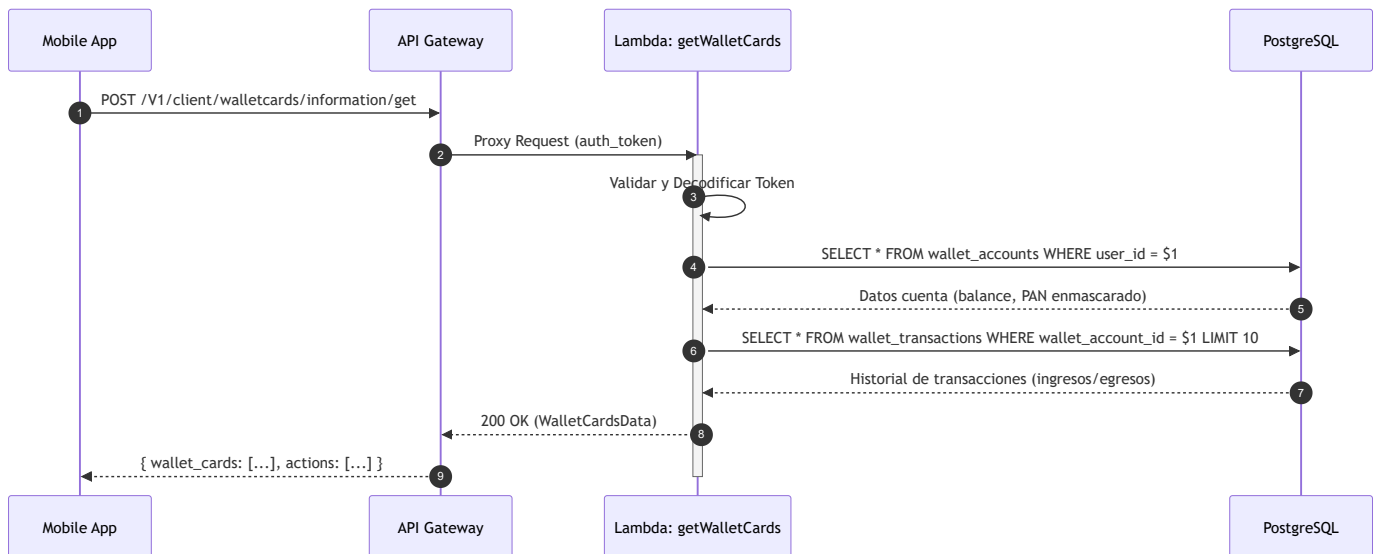
D. Flujo de Validación Onboarding (validateUser & validateOtp)

Fase temprana de onboarding donde se consultan listas grises/negras, persistencia parcial y confirmación SMS (OTP).



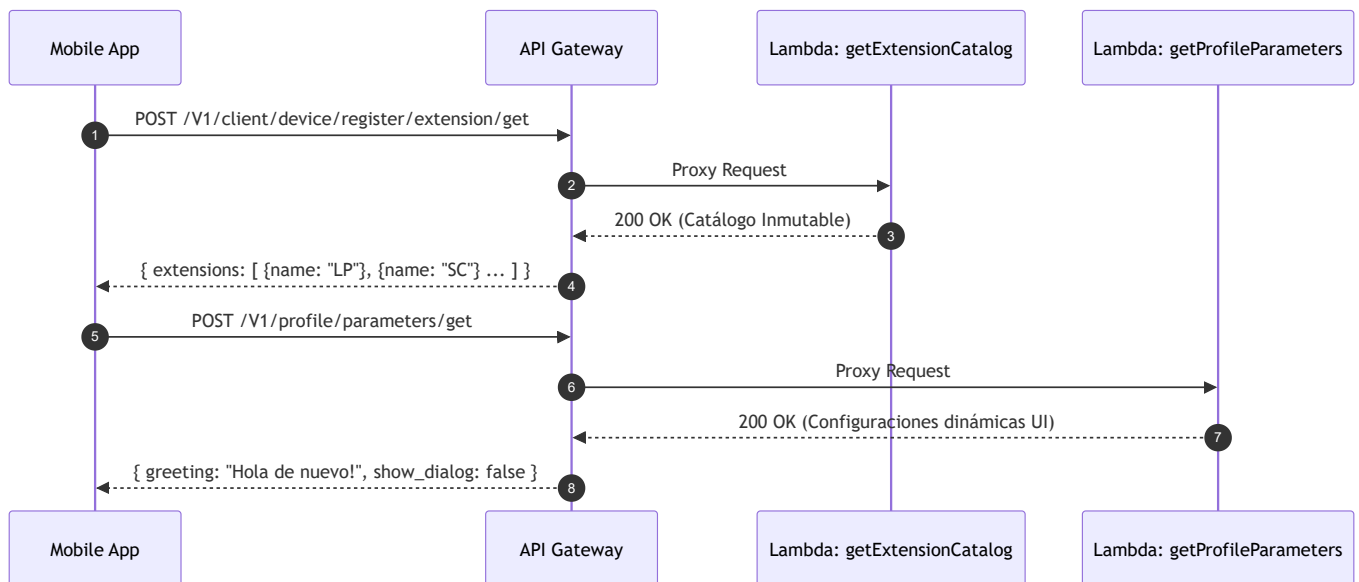
E. Flujo de Consulta Financiera (GetWalletCards)

Obtiene información consolidada de los balances financieros y últimas transacciones en el Home del App.



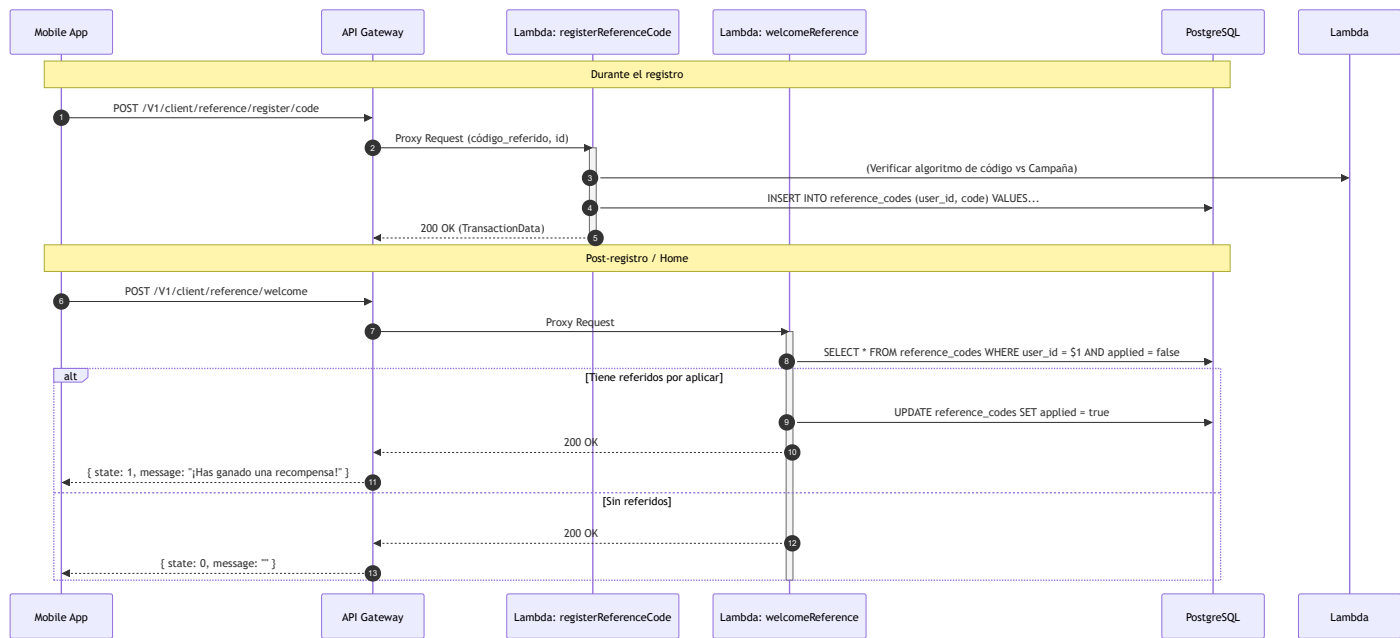
F. Metadatos, Catálogos y Parámetros (GetExtensionCatalog & GetProfileParameters)

Endpoints estáticos optimizados (potencial uso de caché distribuido futuro) para configuración de la UI.



G. Flujo de Captación (Referidos: RegisterReferenceCode & WelcomeReference)

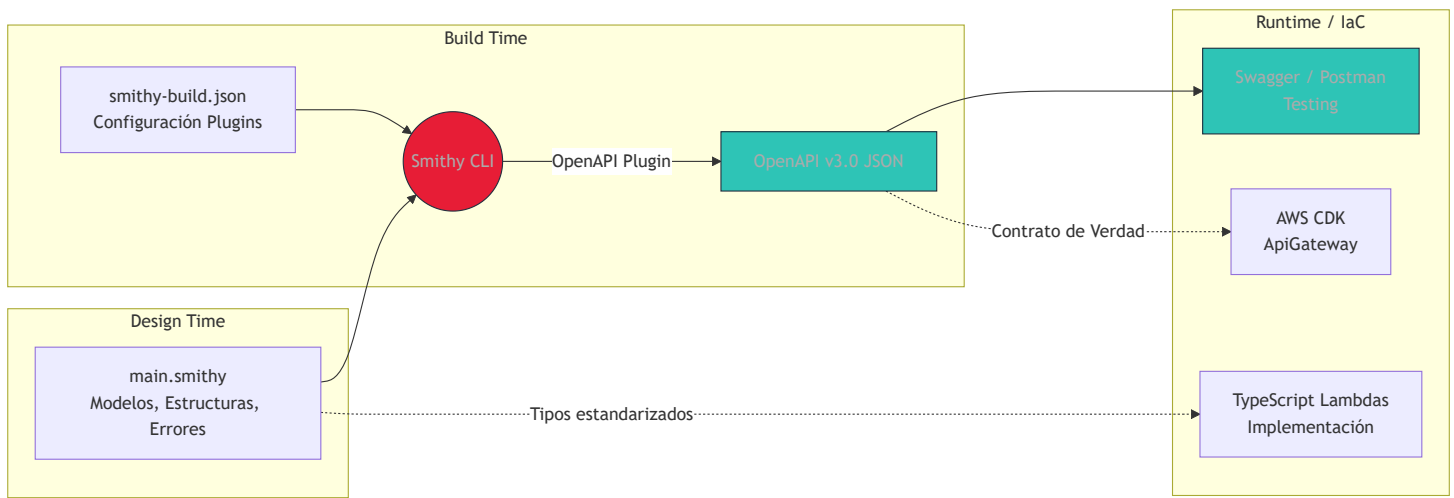
Programa de afiliación y recompensa por onboarding exitoso con código promocional.



Nota del Arquitecto: Este repositorio consolida la infraestructura como código junto con la lógica de negocio, creando una cápsula autónoma (Monorepo). La separación por capas facilita escalar el modelo a arquitecturas de microservicios o integraciones con sistemas core bancarios en el futuro sin reestructurar los contratos definidos en Smithy.

7. Arquitectura de Modelado API (AWS Smithy)

El proyecto utiliza una canalización automatizada para convertir modelos abstractos de dominio en documentación y contratos estrictos (Generation Architecture).



8. Gestión del Proyecto: Product Backlog y Sprints

El desarrollo de la arquitectura y el backend se ejecutó mediante un marco de trabajo ágil. A continuación, se presenta el backlog retrospectivo del proyecto agrupado en historias de usuario y organizado en Sprints de implementación para llegar a su estado actual.

Product Backlog (Épicas)

- **EP-01:** Infraestructura como Código (VPC, RDS, APIGW).
- **EP-02:** Gestión de Identidades y Dispositivos (Cognito, Device Trust).
- **EP-03:** Onboarding Financiero (Validación, Biometría, Creación).
- **EP-04:** Operaciones de Cliente (Login, Wallet, Referidos).

Planificación de Sprints

Sprint 1: Fundación Arquitectónica y Contratos

Objetivo: Establecer el monorepo, los modelos agnósticos y la infraestructura de red y datos.

- **HU-1.1:** *Como Arquitecto, quiero definir el contrato de la API en AWS Smithy para estandarizar payloads y errores.*
 - Tarea 1: Instalar y configurar dependencias de AWS Smithy en `smithy-build.json`.
 - Tarea 2: Modelar los 13 endpoints usando el protocolo `restJson1`.
- **HU-1.2:** *Como DevOps, quiero crear un proyecto CDK con VPC y Base de datos RDS PostgreSQL.*
 - Tarea 1: Inicializar el App de CDK y crear `DatabaseStack` con VPC privada aislada.
 - Tarea 2: Configurar el esquema relacional DDL (`schema.sql`) de tablas transaccionales.
 - Tarea 3: Integrar `SecretsManager` para contraseñas rotativas del RDS.

Sprint 2: Seguridad y Componentes Base

Objetivo: Desplegar IAM, Amazon Cognito y la arquitectura base de integración.

- **HU-2.1:** *Como DevSecOps, quiero provisionar Amazon Cognito para gestionar el ciclo de vida de usuarios.*
 - Tarea 1: Programar `AuthStack` definiendo User Pools con alias en número de teléfono.
 - Tarea 2: Implementar atributos personalizados de Cognito (CIC, Documento de Identidad).
- **HU-2.2:** *Como Backend Developer, quiero crear una capa de conexión reutilizable a PostgreSQL.*
 - Tarea 1: Programar abstracción singleton de `pg.Pool` con manejo de latencia en "warm-starts".
 - Tarea 2: Consumir credenciales seguras vía `SecretsManagerClient` de AWS SDK v3.

Sprint 3: Flujo Crítico de Onboarding

Objetivo: Implementar los handlers Lambda necesarios para registrar un usuario de extremo a extremo.

- **HU-3.1:** *Como Usuario Nuevo, quiero validar mi documento y teléfono (OTP) para continuar mi registro.*
 - Tarea 1: Desarrollar Lambda `validateUser` consultando usuarios pre-existentes en la Base de Datos.
 - Tarea 2: Codificar el manejador lógico para `validateOtp`.
- **HU-3.2:** *Como Usuario Validado, quiero que mi cuenta financiera y digital sea creada.*
 - Tarea 1: Desarrollar handler Lambda `createAccount`.
 - Tarea 2: Coordinar transacción distribuida: Insertar Identidad en Cognito + Insertar ACID en tablas `users`, `devices` y `wallet_accounts` en RDS.

Sprint 4: Biometría y Dispositivos de Confianza

Objetivo: Implementar la prueba biométrica (Liveness) y asociar seguridad de hardware.

- **HU-4.1:** *Como Sistema de Seguridad, quiero asegurar que el dispositivo sea registrado y confiable.*
 - Tarea 1: Programar endpoint `deviceIdentification` para generar un ID criptográfico.
 - Tarea 2: Construir flujo `deviceAuthenticate` para re-ingresos con hardware conocido.
- **HU-4.2:** *Como Oficial de Fraude, quiero que el usuario apruebe validación facial antes de entrar al sistema.*
 - Tarea 1: Programar Lambda `initFaceRecognition` registrando la sesión temporal en BD.
 - Tarea 2: Desarrollar `executeFaceRecognition` para recibir el token base64 y cambiar el estatus a "VERIFIED".

Sprint 5: Módulo de Cliente y Fidelización

Objetivo: Dar soporte al inicio de sesión, visualización del balance financiero y campañas.

- **HU-5.1:** *Como Cliente Registrado, quiero iniciar sesión en mi aplicación para consultar mi balance (Wallet).*
 - Tarea 1: Programar la validación `login` ejecutando el flujo `USER_PASSWORD_AUTH` contra AWS Cognito.
 - Tarea 2: Programar Lambda `getWalletCards` para cruzar movimientos y saldo en PostgreSQL.
- **HU-5.2:** *Como Equipo de Growth, quiero habilitar recompensas por referidos.*
 - Tarea 1: Programar endpoints `registerReferenceCode` y `welcomeReference`.
 - Tarea 2: Diseñar y codificar `getProfileParameters` para el dinamismo de la UI.

9. Estrategia y Ejecución de Pruebas (Quality Assurance)

El proyecto cuenta con un enfoque pragmático orientado a **Pruebas de Integración End-to-End (E2E)** ejecutadas directamente contra los entornos desplegados en AWS. Esto garantiza que la configuración de API Gateway, los permisos IAM de las Lambdas, la validación en Cognito y las transacciones en RDS funcionen como un ecosistema cohesionado.

Automatización de Pruebas (Scripts)

Para la validación continua y prevención de regresiones, se construyeron rutinas automatizadas en PowerShell ubicadas en la raíz del proyecto:

- `test-endpoints.ps1` : Simula el flujo completo de Onboarding, mock de biometría y aprovisionamiento de cuenta de extremo a extremo.
- `test-login.ps1` / `test-login2.ps1` : Valida los flujos de autenticación recurrente, interceptación de tokens y manejo de errores con Cognito.

Casos de Prueba Críticos (Test Cases)

A continuación, se documentan los Casos de Prueba (TC) principales que rigieron la certificación del backend:

ID	Caso de Prueba	Precondiciones	Pasos de Ejecución	Resultado Esperado	Estado
TC-01	Identificación de Nuevo Dispositivo	Ninguna.	1. Hacer POST a <code>/v1/device/identification</code> con un UUID de hardware simulado. 2. Validar respuesta HTTP.	Código 200. Retorna <code>certified_id</code> , <code>auth_token</code> e inserta fila en RDS <code>devices</code> .	 Pasó
TC-02	Validación de Identidad (Onboarding)	El documento no debe existir en RDS.	1. POST a <code>/v1/register/validate/user</code> enviando DNI y email.	Código 200. El payload <code>data</code> debe retornar vacío indicando que es libre para registro.	 Pasó
TC-03	Validación de Identidad (Rechazo)	El documento YA existe en RDS.	1. POST a <code>/v1/register/validate/user</code> enviando DNI existente.	Código 200. Retorna la información enmascarada para prevenir duplicación.	 Pasó
TC-04	Inicialización Biométrica	Documento válido enviado.	1. POST a <code>/v1/register/init/face/recognition</code> . 2. Extraer <code>session_id</code> .	Código 200. Se crea sesión temporal en <code>face_sessions</code> con status <code>PENDING</code> .	 Pasó
TC-05	Creación y Transaccionalidad de Cuenta	Pines generados, OTP validado.	1. POST a <code>/v1/register/create/account</code> con PIN, celular y datos. 2. Verificar Amazon Cognito. 3. Verificar RDS.	Código 200. Usuario es creado en Cognito. Las tablas <code>users</code> , <code>devices</code> y <code>wallet_accounts</code> se enlazan atómicamente.	 Pasó
TC-06	Autenticación (Login Exitoso)	Cuenta creada en TC-05.	1. POST a <code>/v1/client/login/get</code> con celular (Alias) y PIN. 2. Interceptar tokens JWT.	Código 200. Devuelve el JWT encapsulado en <code>private_token</code> e información del Perfil RDS.	 Pasó
TC-07	Prevención de Login Inválido	Cuenta existente.	1. POST a <code>login/get</code> con PIN erróneo repetidamente.	Cognito detecta falla (<code>NotAuthorizedException</code>), API Gateway retorna 401/400.	 Pasó
TC-08	Consulta de Wallet y Movimientos	Autenticado (Token válido).	1. POST a <code>/v1/client/walletcards/information/get</code> .	Retorna balance numérico, número de cuenta virtual y lista de transacciones.	 Pasó

Evaluación QA: Las pruebas demuestran que las políticas de consistencia eventual (Cognito) y consistencia fuerte (ACID en RDS) coexisten sin generar estados huérfanos. Las barreras de autorización protegen exitosamente las rutas privadas.

Arquitectura y Flujos de Automatización de Pruebas

Para garantizar que los flujos se prueban en un entorno idéntico al de producción, la arquitectura de pruebas ejecuta rutinas (PowerShell) desde el entorno local contra el API Gateway remoto, inyectando mocks de información en cada invocación.

Diagrama de Arquitectura de Testing

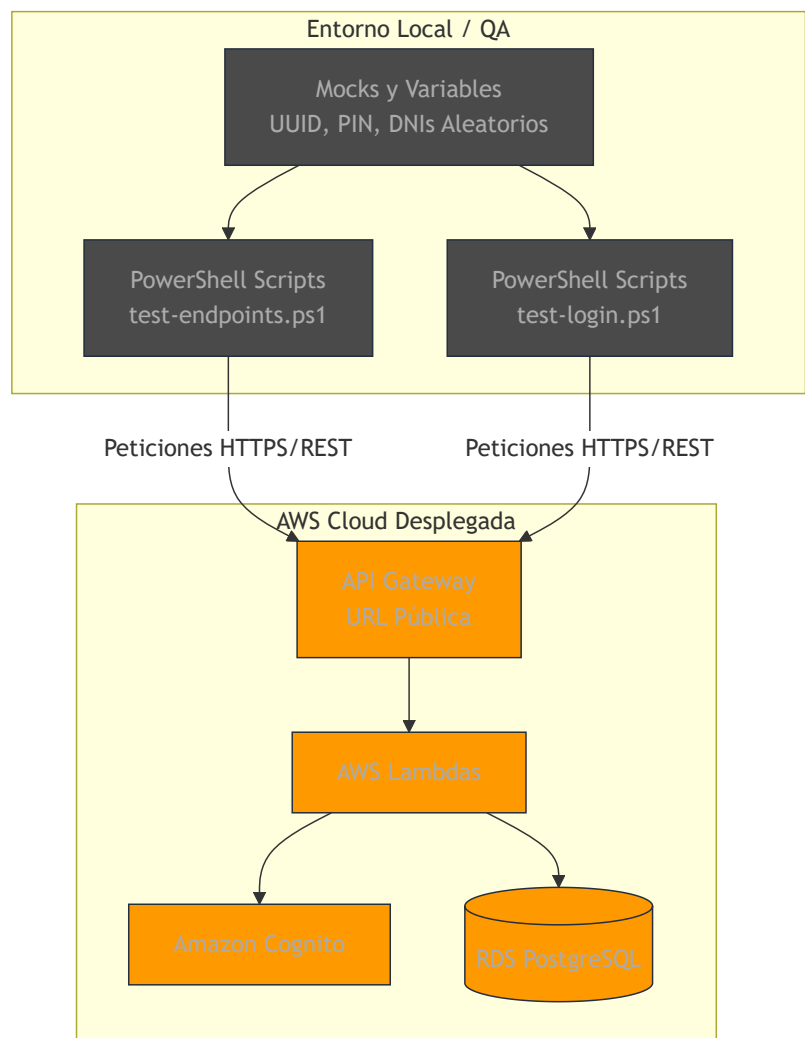


Diagrama de Secuencia: Flujo E2E de Onboarding (test-endpoints.ps1)

Este script orquesta una prueba de integración (Integration Testing) encadenando múltiples llamados, asegurando que el estado generado por un endpoint es correctamente consumido por el siguiente.

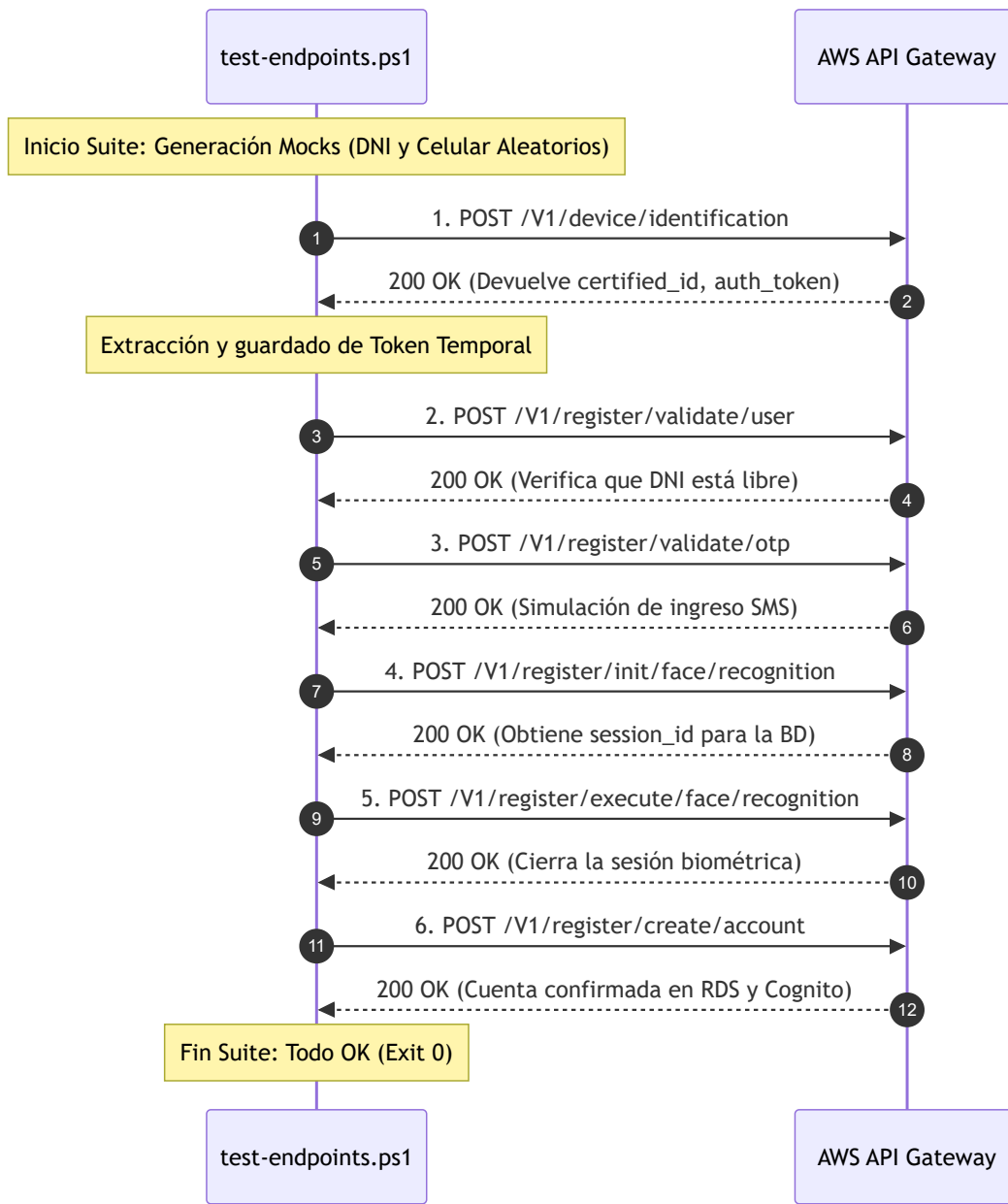


Diagrama de Secuencia: Flujo E2E de Autenticación (test-login.ps1)

Este script se enfoca en las pruebas de autenticación repetitiva, validando el correcto intercambio de credenciales por Tokens JWT reales emitidos por AWS Cognito y el perfil de RDS.

